

Cẩm nang đọc source và lần theo pipeline VideoLevel

Từ đầu vào H.264 đến proof, nhúng, blind extraction và xác minh

Tài liệu kỹ thuật tiếng Việt

26 tháng 9 năm 2026

Mục lục

1	Mục đích và cách dùng tài liệu	3
1.1	Ba lớp kết quả cần phân biệt	3
1.2	Một khác biệt kiến trúc ảnh hưởng đến giao thức	3
2	Sơ đồ hoạt động tổng thể	3
2.1	Luồng gọi theo thứ tự	3
3	Cách đọc cấu trúc repository	4
4	Pha prover: từ message đến proof-bearing payload	5
4.1	Chuẩn bị session: script 00	5
4.2	Các helper dùng chung trong demo	5
4.3	Circuit hiện đang chứng minh điều gì	6
4.4	Ví dụ byte-for-byte với demo	7
4.5	Mã hóa/nén proof	7
5	Đường native: đọc H.264 và chọn vị trí sign CAVLC	7
5.1	Tại sao sửa đúng một bit hệ số có thể giữ parser đồng bộ	7
5.2	Annex-B, EBSP và RBSP	7
5.3	Các kiểu dữ liệu C++ giữ kết quả parser	8
5.4	Call chain C++ từ Annex-B đến candidate	8
5.5	Ví dụ một candidate	9
6	Lịch keyed, frame integrity và stream encoder/decoder	9
6.1	Định danh candidate và thứ tự bí mật	9
6.2	Authenticated frame bên trong stream	9
6.3	Patch và extraction bit-level	10
6.4	CLI native và chế độ byte stream	10
7	Verifier: từ video stego đến quyết định accept/reject	11
7.1	Blind extraction native	11
7.2	Unpack và verify Groth16	11

7.3	HTTP và WebSocket làm gì, không làm gì	11
8	API chung: hàm nào nhận request, hàm nào chạy job	12
9	Nhánh Python full-video cũ: các hàm và phụ thuộc	13
9.1	Không nhầm nhánh này với native blind	13
9.2	Các helper của embedder	13
9.3	Phân tích H.264/CAVLC của Python path	13
9.4	Safety filter, patch và tái dựng Python	14
9.5	Verifier Python yêu cầu những gì	15
10	Cache, scheduler và những nhánh hỗ trợ	15
10.1	Cache phân tích video	15
10.2	Realtime scheduler abstraction	15
10.3	Key certificate và manifest	15
11	Demo đã chạy: input cụ thể và cách suy luận	16
11.1	Artifacts nên giữ khi debug	16
12	Các giới hạn cần nhớ khi đọc code	17
13	Lộ trình đọc source cho người mới tiếp nhận	17
14	Tra cứu nhanh theo câu hỏi debug	17
15	Kết luận vận hành	18

1 Mục đích và cách dùng tài liệu

Tài liệu này là bản đồ đọc code của repository VideoLevel. Mục tiêu là giúp người đọc lần theo một dữ liệu cụ thể qua từng lớp: script demo, giao thức API, chương trình C++ phân tích CAVLC, bộ đóng gói Groth16 và bộ xác minh. Mỗi phần nêu file, hàm, đầu vào, đầu ra, tác dụng lên pipeline và bước kế tiếp sử dụng kết quả đó như thế nào. Đây là hướng dẫn source, không phải đặc tả cho mọi profile H.264.

Điểm quan trọng đầu tiên là repository hiện có hai đường xử lý video khác nhau. Demo chính từ bước 00 đến 12 gọi CLI native C++ để nhúng/trích xuất theo lịch blind có khóa. Script 13 chạy riêng nhánh Python cũ `src/embedder.py` / `src/verifier.py`; nhánh này phân tích toàn video, dùng sidcar/vị trí và có chi phí lớn hơn. Không nên lấy tên hàm Python `embed()` làm bằng chứng rằng HTTP/WebSocket realtime hiện dùng hàm đó: native adapter gọi `zkstego_blind_bits`.

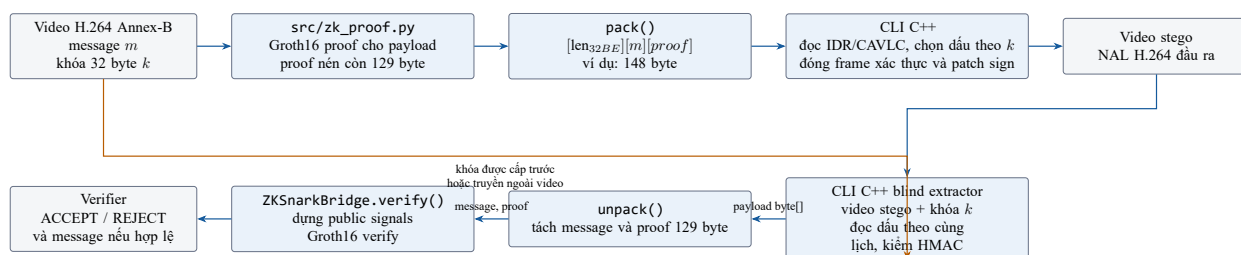
1.1 Ba lớp kết quả cần phân biệt

- **Thành công cú pháp video:** FFmpeg đọc và giải mã stream mà không báo lỗi nghiêm ngặt. Điều này chưa đo PSNR, SSIM hay chất lượng cảm nhận.
- **Thành công khôi phục payload:** extractor lấy đúng các byte đã đóng gói từ dấu hệ số CAVLC trong video. Điều này cần đúng khóa của lịch blind và frame authentication.
- **Thành công mệnh đề ZK:** verifier kiểm tra Groth16 proof trên đúng public inputs. Kết quả này không tự chứng minh nguồn gốc camera, danh tính người ký, thời hạn giấy phép hay video không bị xử lý ở nơi khác.

1.2 Một khác biệt kiến trúc ảnh hưởng đến giao thức

Video stego là artifact mang thông điệp và proof, nhưng trong cài đặt hiện tại verifier còn cần khóa 32 byte để blind-extract và để helper Python dựng public signals. Nếu verifier đã được cấp khóa an toàn từ trước thì media channel có thể chỉ truyền video; nếu chưa, cần một kênh cấp khóa riêng. Endpoint native `/api/v1/jobs/verify` hiện bị cấu hình thành 501: native API `embed/extract` không tự chạy Groth16 verifier. Demo thực hiện Groth16 verification ở bước Python riêng sau khi trích xuất.

2 Sơ đồ hoạt động tổng thể



Hình 1: Luồng proof-bearing của demo. Khóa là input ngoài video ở cả pha nhúng lẫn blind extraction.

2.1 Luồng gọi theo thứ tự

Bước	Điểm vào	Input chính	Output chuyển tiếp
00	<code>demo/00_prepare.py</code>	Đường dẫn cover H.264, message text	<code>session.json</code> , <code>message.bin</code> , <code>secret_key.bin</code>

Bước	Điểm vào	Input chính	Output chuyển tiếp
01	demo/01_generate_proof.py	message.bin, secret_key.bin, artifacts circuit	proof JSON, proof nén 129 B, public signals
02	demo/02_pack_payload.py	message.bin, proof nén	packed_payload.bin
03–04	demo/03_read_video.py, 04_find_positions.py	Annex-B H.264, kích thước payload	log parser, số IDR/candidate/capacity, probe một sign
05	demo/05_embed_video.py	Cover, packed payload, key	stego_video.h264
06–07	demo/06_read_stego_video.py, 07_blind_extract.py	Stego, FFmpeg, key	log decode, extracted_payload.bin
08–09	demo/08_unpack_payload.py, 09_verify_proof.py	Payload, verification key, key	message/proof khôi phục và kết quả Groth16
10–12	Wrong-key, HTTP job, WebSocket E2E	Video/payload/key và channel test	Log từ chối, round-trip, test report
13 (tùy chọn)	demo/13_python_pipeline.py	Contract benchmark Python, video khác, key	python_pipeline_stego.h264, cache, log
14	demo/14_summary.py	Các log hiện có và session.json	SUMMARY.txt

3 Cách đọc cấu trúc repository

Đường dẫn	Vai trò và ảnh hưởng đến pipeline
demo/00...14_*.py	Kịch bản có thể chạy theo từng công đoạn. Chúng ghép các module hiện có; không phải mỗi file là một implementation codec riêng.
src/zk_proof.py	Định nghĩa payload format, serialize proof Groth16, sinh witness/proof bằng Node/snarkjs, dựng public inputs và verify.
circuits/payload_verify.circom	Mệnh đề circuit: quan hệ SHA-256 giữa hash payload, commitment và secret; đây là logic được proving key chứng minh.
native/tools/blind_bits.cpp	CLI native: đọc key/payload, chọn chế độ, stream H.264, gọi API trong thư viện C++, ghi H.264 hoặc payload.
native/src/cavlc_stream.cpp	Parser Annex-B/SPS/PPS/IDR/CAVLC, sinh candidate sign, lịch keyed, framing HMAC, encoder/decoder incremental.
native/include/zkstego/cavlc_stream.hpp	Kiểu dữ liệu và interface công khai của C++: SPS/PPS/slice, macroblock, candidate, stream reader/encoder/decoder.
src/api/native_handlers.py	Adapter FastAPI/WebSocket gọi native CLI; quản lý input streaming, bounded buffers, resource metrics và HTTP handlers.
src/api/app.py	Lỗi API chung: settings, auth token, giới hạn request/upload, queue/job store, file lưu tạm/cleanup, endpoint job.
src/embedder.py, src/verifier.py	Nhánh Python full-video cũ; phân tích cover, chọn/ghi sidecar vị trí, patch/reconstruct và verify theo cover.

Đường dẫn	Vai trò và ảnh hưởng đến pipeline
src/bitstream/	Parser Python H.264/CAVLC và bit reader/writer; chủ yếu được dùng bởi nhánh Python, không phải parser native gọi từ FastAPI.
src/core/pipeline.py, src/core/stego.py	Phân tích IDR, vị trí safe, sửa/trích bit và dựng video cho nhánh Python; một số iterator streaming cũng được các prototype blind dùng.
src/native_blind_contract.py	Python executable specification của serialization candidate, HMAC schedule và frame native. Đây là contract/test aid; native CLI vẫn thi hành logic C++.
src/realtime_cavlc.py	Scheduler abstraction có giới hạn queue/latency. Tests exercise abstraction; native WebSocket adapter hiện tự quản lý subprocess/backpressure chứ không gọi scheduler này.
src/key_policy.py, src/manifest.py	Certificate thời hạn Ed25519 và sidecar manifest/signature. Đây không được gọi bởi demo native HTTP/WebSocket hiện tại; không nên kết luận nó tự hết hạn khóa native nếu chưa tích hợp tại enforcement point.
src/trust/, src/provenance/	Các helper fingerprint/attestation/provenance mở rộng. Không nằm trên luồng 00–12 đã demo; xem test/API cụ thể trước khi coi là runtime trust root.

4 Pha prover: từ message đến proof-bearing payload

4.1 Chuẩn bị session: script 00

demo/00_prepare.py::main() parse cờ --input và --message, đọc toàn bộ cover để tính byte length/SHA-256, tạo folder demo/runs/session_<UTC>/, ghi message UTF-8 thành message.bin và sinh khóa demo ngẫu nhiên 32 byte vào secret_key.bin với quyền hạn chế. session.json lưu đường dẫn input, số byte, hash và thời điểm; log không ghi nội dung khóa.

Output này điều khiển mọi bước sau. Nếu đường dẫn --session không đúng, common.require_session() dừng trước khi module proof/native được gọi. Việc truyền /runs/... trên Windows tạo đường dẫn từ gốc ổ đĩa; từ thư mục repository phải dùng đường dẫn đầy đủ hoặc demo/runs/....

4.2 Các helper dùng chung trong demo

Mỗi file đánh số đều có main(), nhưng phần lớn chỉ điều phối: đọc --session, kiểm artifact đầu vào, gọi module thật rồi in đường dẫn/log. Các hành vi dùng chung nằm trong demo/common.py:

Hàm	Input → output	Vai trò trong stage
require_session()	Path argument → session folder + state dict	Resolve path, đọc session.json, dừng sớm nếu session hỏng/không tồn tại.
read_key()	Session → 32-byte key	Đọc key ở bước native/ZK; không ghi bytes khóa vào log.
native_tool()	Tên CLI → built executable path	Kiểm tra tìm thấy binary native đúng tên trước subprocess.

Hàm	Input → output	Vai trò trong stage
<code>require_artifact()</code>	Session + filename → existing path	Làm dependency gate giữa các bước, báo file trung gian còn thiếu.
<code>write_process_log()</code>	command/stdout/stderr/exit → text log	Lưu lệnh, mô tả stdin đã redacted, streams và mã thoát để debug.
<code>run_logged()</code>	Native command + optional stdin → completed process result	Chạy subprocess có timeout, lưu output/log; key stdin được mô tả redacted.
<code>python_stage_log()</code>	Stage/title/lines → stage log	Ghi marker kết quả cho script Python không có process con.
<code>run_python_logged()</code>	Callable Python → PASS/error log	Bắt exception, ghi traceback/status và cho stage dừng đúng mã lỗi.

`demo/test_common.py::test_stage_log_records_full_process_result_without_stdin_secret()` kiểm tra log chứa đầy đủ kết quả process mà không lộ secret stdin. Các script 00–14 và artifact cụ thể của chúng được liệt kê ở Bảng 1; file test này chỉ kiểm tra tiện ích log, không sinh artifact của một lần chạy pipeline.

4.3 Circuit hiện đang chứng minh điều gì

Trong `src/zk_proof.py`, payload là bytes p và khóa 32 byte là k . `ZKSnarkBridge._build_circuit_input()` tính

$$h = \text{SHA256}(p), \quad c = \text{SHA256}(h\|k).$$

Circuit `circuits/payload_verify.circom` nhận h , c và độ dài payload làm public signals, còn k là private witness; constraints kiểm tra rằng c đúng là hash của $h\|k$ và độ dài nằm trong miền hợp lệ. Groth16 proof là bằng chứng cho quan hệ này dưới circuit/proving key hiện hành. Circuit này không nhận pixel/frame H.264 và không chứng minh video xuất phát từ camera hay rằng video chưa bị transcode.

Trong demo, `01_generate_proof.py` gọi `generate_proof_for_payload(message, key)` với đúng bytes trong `message.bin`. Vì vậy proof ràng buộc message user (15 byte trong session mẫu), không phải toàn bộ blob 148 byte đã gồm proof; bước 02 mới tạo outer blob để chuyển qua H.264.

Hàm/file	Input → output	Tác dụng kế tiếp
<code>_build_circuit_input(p, k)</code>	Bytes p , key 32 B k → dictionary các bit h, c, k và length	Node witness generator đọc JSON này.
<code>_compute_witness(input)</code>	Circuit input → file witness tạm	Gọi JS/WASM; xóa file input chứa key sau khi tạo witness.
<code>_snarkjs_prove(witness)</code>	Witness + proving key → proof JSON và public JSON	Trả về hai đối tượng snarkjs cho bước serialize và demo log.
<code>generate_proof_for_payload(p, k)</code>	Payload bytes + key → (<i>proof</i> , <i>public</i>)	Đầu vào công khai ở đây không tự được gửi qua C++ embed; proof mới được serialize thành bytes.
<code>proof_to_bytes(proof)</code>	Pi A/B/C dạng point → 129 bytes	Cắt kích thước proof JSON để nhúng vào elementary stream.

Hàm/file	Input → output	Tác dụng kế tiếp
<code>pack(p, proofBytes)</code>	$p, \text{ proof } 129 \text{ B} \rightarrow [4\text{-byte len}_{BE}[p][proof]]$	Blob được gửi cho native embed như payload opaque.
<code>blob_bit_length(p)</code>	Độ dài message $\rightarrow (4 + p + 129) \times 8 \text{ bit}$	Ước lượng capacity ở các caller Python cũ.

4.4 Ví dụ byte-for-byte với demo

Với message `ZKdemopayload`, UTF-8 dài 15 byte. Sau khi sinh proof, `proof_to_bytes()` trả về đúng 129 byte. `pack()` ghép:

```
00 00 00 0f | 5a 4b 20 64 65 6d 6f 20 70 61 79 6c 6f 61 64 | <129 proof bytes>
length=15                                     message UTF-8
```

Tổng blob là $4 + 15 + 129 = 148$ byte, tương đương 1.184 bit. Blob 148 byte này chính là đầu vào payload mà script 05 đưa cho native CAVLC embedder.

4.5 Mã hóa/nén proof

`proof_to_bytes()` ghi bốn tọa độ x của các điểm Groth16 (mỗi tọa độ 32 byte big-endian) và một byte cờ chứa parity/sign của tọa độ y . Tổng là $4 \times 32 + 1 = 129$ byte. `bytes_to_proof()` làm chiều ngược: khôi phục sign, giải phương trình đường cong BN128 để dựng lại y , rồi tạo cấu trúc `pi_a`, `pi_b`, `pi_c` mà snarkjs nhận.

Đây là serialization riêng của source. Không được cắt/gắn proof tùy ý: mọi byte đều đi vào phép giải nén tọa độ hoặc cờ sign, và sai định dạng làm verify thất bại hoặc bị assertion/error.

5 Đường native: đọc H.264 và chọn vị trí sign CAVLC

5.1 Tại sao sửa đúng một bit hệ số có thể giữ parser đồng bộ

CAVLC residual block mở đầu bằng `coeff_token` để mã hóa `TotalCoeff` và `TrailingOnes`. Với mỗi trailing one, stream ghi một sign bit riêng: bit 0 nghĩa giá trị hệ số dương 1, bit 1 nghĩa âm 1. Đổi sign chỉ thay giá trị hệ số từ +1 sang -1 hoặc ngược lại; không đổi số hệ số, số zero-run, VLC codeword length hay vị trí bắt đầu block tiếp theo. Vì vậy native implementation tìm sign position đã parse được rồi patch đúng RBSP bit đó, thay vì decode frame thành pixel rồi encode lại.

Điều này chỉ an toàn cho candidate sign hợp lệ. Không thể lật tùy tiện một bit trong NAL: nó có thể sửa `coeff_token`, level, macroblock header hoặc emulation prevention byte và khiến các bit sau bị đọc lệch.

5.2 Annex-B, Ebsp và Rbsp

`split_annex_b()` quét start code 3/4 byte, cắt elementary stream thành NAL units và giữ header/index/offset. `AnnexBNalUnit::rbsp()` gọi `ebsp_to_rbsp()` để bỏ emulation-prevention byte `0x03` đặt sau `0000`. Parser đọc syntax trên RBSP. Khi đã patch xong, `rbsp_to_ebsp()` chèn prevention bytes cần thiết và `assemble_annex_b()` ghép start code + header + payload về Annex-B.

Ví dụ nếu trong Ebsp có chuỗi byte mã hóa tương đương `000001`, encoder phải ngăn chuỗi đó bị nhầm với start code; parser trước hết bỏ byte escape, đọc bit syntax trên RBSP, sau đó serializer phục hồi escape khi ghi output.

5.3 Các kiểu dữ liệu C++ giữ kết quả parser

Kiểu trong header	Dữ liệu và consumer tiếp theo
AnnexBNalUnit	Offset, start-code length, NAL header/type, EBSP payload; dùng để tìm IDR, parameter set và RBSP.
H264BaselineSps	Profile/level, kích thước picture theo macroblock, frame/POC fields; xác định cấu trúc và cách đọc header.
H264BaselinePps	Entropy mode và các syntax flag; native path yêu cầu CAVLC, không phải CABAC.
H264BaselineIdrSliceHeader	first MB, slice type, PPS id, frame/id fields, QP delta và offset residual đầu slice.
CavlcCoeffToken	TotalCoeff, TrailingOnes và offset của sign bits; là nền để giải mã level/tail.
CavlcDecodedLumaBlock	Token, levels, zero/run tail, hệ số tái dựng; consumer tạo sign candidates.
CavlcDecodedIMacroblock	Header I-macroblock và residual plane blocks; slice decoder đi qua macroblock theo địa chỉ.
CavlcDecodedIdrSlice	SPS/PPS/slice header, macroblocks và trailing-bit offset; đầu vào collector candidate.
CavlcSignCandidate	(NAL index, macroblock address, category, block index, RBSP bit offset); identity vật lý sẽ được xếp thứ tự bằng key.
AuthenticatedCavlcStreamSegment	Bytes output, candidate capacity, bit count đã nhúng, trạng thái complete; trả về qua mỗi segment.

5.4 Call chain C++ từ Annex-B đến candidate

Các hàm dưới đây nằm chủ yếu trong `native/src/cavlc_stream.cpp`, interface ở `native/include/zkstego/cavlc_stream.hpp`:

Hàm	Input → output	Ảnh hưởng/consumer
<code>ebsp_to_rbsp() / rbsp_to_ebsp()</code>	Byte vector ↔ byte vector	Chuyển giữa byte syntax NAL bên ngoài và bitstream RBSP parser/patcher thao tác.
<code>parse_baseline_sps()</code>	SPS RBSP → SPS struct	Cung cấp dimensions/flags cho slice và MB parsing.
<code>parse_baseline_pps()</code>	PPS RBSP → PPS struct	Cung cấp PPS syntax; kiểm entropy mode và parameters.
<code>parse_baseline_idr_slice_header()</code>	Slice RBSP + SPS/PPS → header, data offset	Chỉ ra điểm bắt đầu macroblock residual.
<code>parse_cavlc_coeff_token()</code>	RBSP, start bit, n_C → TotalCoeff/TrailingOnes/sign offsets	Chọn bảng VLC đúng context; trả candidate sign offsets.
<code>decode_cavlc_non_trailing_levels()</code>	Token + RBSP → trailing values, levels, next offset	Nhảy qua phần level sau sign bits để xác định tail đúng.

Hàm	Input → output	Ảnh hưởng/consumer
<code>decode_cavlc_residual_tail()</code>	TotalCoeff, giới hạn block, offset → total zeros/runs/next offset	Đồng bộ vị trí block kế tiếp.
<code>reconstruct_cavlc_block()</code>	Levels + runs + max coefficients → vector coefficients	Xác nhận residual có cấu trúc phù hợp và điền vị trí coefficient.
<code>decode_cavlc_luma_block()</code> <code>/ decode_cavlc_chroma_*</code>	RBSP + offset + context → decoded block	Chuyên biệt hóa luma/chroma, luma AC/DC.
<code>decode_cavlc_luma_macroblock()</code>	CBP, offset, neighbour TotalCoeff → 16 luma blocks	Tính n_C theo hàng xóm đã giải mã, xử lý từng block coded.
<code>parse_baseline_i_macroblock_header()</code>	RBSP + offset → MB type, CBP, QP delta, residual offset	MB parser quyết định syntax residual tiếp theo.
<code>decode_baseline_i_idr_slice()</code>	RBSP + SPS/PPS → một decoded IDR slice	Lắp macroblock I slice và gom offset/sign metadata.
<code>decode_baseline_i_idr_slices()</code>	Annex-B bytes → danh sách IDR I slices	Dùng bởi capacity scan, embedding và extraction full-buffer.
<code>collect_cavlc_trailing_one_sign_candidates()</code>	Decoded slices → candidate vector	Mỗi residual block đóng góp các sign offsets trailing-one patchable theo parser.
<code>inspect_baseline_idr_headers()</code>	Annex-B → header/mẫu đầu IDR	Công cụ inspect dùng để giải thích NAL/MB đầu, không quyết định lịch embed cuối.

5.5 Ví dụ một candidate

Candidate trong log inspect demo có thể đọc như địa chỉ trong syntax, không phải tọa độ pixel: `patched_nal=3macroblock=0rbps_bit=91`. Công cụ `zkstego_idr_inspect--flip-first-sign` tạo `candidate_probe.h264` để thấy một sign bit riêng lẻ có thể patch. File probe này không chứa payload hoàn chỉnh và không phải output của bước 05.

6 Lịch keyed, frame integrity và stream encoder/decoder

6.1 Định danh candidate và thứ tự bí mật

`serialize_cavlc_sign_candidate()` biến tuple candidate thành chuỗi định danh ổn định gồm NAL, MB, loại residual, block và offset. Hàm `score_keyed_cavlc_sign_candidate()` tính HMAC-SHA-256 trên định danh bằng khóa lịch có domain separation. `select_keyed_cavlc_sign_candidates()` sắp xếp candidate theo score rồi lấy đủ số bit cần. Verifier quét cùng video stego, tái tạo cùng candidate list và dùng cùng key, nên trích bit theo đúng thứ tự. Khóa không được giấu trong H.264; nó phải được provision riêng.

6.2 Authenticated frame bên trong stream

Native C++ dùng một framing khác với outer payload format của Python:

```
frame = version_u8 || payload_length_u16_be || payload || tag_16_bytes
```

Tag là 16 byte đầu của HMAC-SHA-256 theo domain riêng. Phần overhead là $1 + 2 + 16 = 19$ byte. Vì vậy blob proof-bearing 148 byte trở thành frame 167 byte, tức 1.336 bit phải nhúng. Đây là lý do log capacity báo 1.336 bit, trong khi `02_pack_payload.log` báo 1.184 bit.

`pack_authenticated_cavlc_frame()` thêm version/length/tag; `unpack_authenticated_cavlc_frame()` kiểm version, giới hạn length, đúng kích thước và so sánh tag constant-time trước khi trả payload. HMAC không thay Groth16: HMAC bảo vệ framing/channel với shared key; Groth16 kiểm quan hệ payload/key theo circuit. Wrong key có thể hỏng keyed schedule và/hoặc HMAC frame, trước cả khi đến bước Groth16.

6.3 Patch và extraction bit-level

`embed_keyed_cavlc_sign_bits()` nhận Annex-B, key, bit vector; giải mã IDR, tạo candidate, chọn đúng số sign offsets và patch sign bit trong RBSP. Hàm không re-encode nguyên video. `extract_keyed_cavlc_sign_bits()` tái tạo candidate list và đọc dấu theo cùng thứ tự. Các helper `bytes_to_msb_bits()` / `msb_bits_to_bytes()` quy định bit order MSB trước để hai phía đồng bộ.

Ở chế độ stream, `AuthenticatedCavlcStreamEncoder::process_segment()` nhận một Annex-B segment (SPS/PPS cộng IDR nếu có), lấy tối đa cấu hình bit mỗi IDR, patch phần kế tiếp của framed payload và báo `bits_embedded` cùng trạng thái complete. `complete()` / `remaining_bits()` hỗ trợ caller biết khi nào payload xong. Decoder dùng `AuthenticatedCavlcStreamDecoder::consume_segment()` để tích lũy sign bits, đọc header length, chờ đủ body+tag rồi chỉ xuất `authenticated_payload()` sau khi kiểm tag; `failed()` là trạng thái fail-closed.

6.4 CLI native và chế độ byte stream

File `native/tools/blind_bits.cpp` là process boundary mà Python API gọi. `main()` dispatch lệnh như `measure-live-capacity-stdin`, `embed-stream-auth-stdin`, `extract-stream-auth` và `extract-live-auth-stdin`. Các helper chính:

Hàm	I/O	Vai trò
<code>decode_hex()</code>	Hex key/payload → sensitive bytes	Kiểm tra ký tự hex và tránh in secret.
<code>read_key_file()</code> / <code>read_key_stdin_line()</code>	Key source → key bytes	Hỗ trợ file quyền hạn chế hoặc truyền key qua stdin, không argv.
<code>read_binary()</code> / <code>write_new_binary()</code>	File bytes ↔ vector	I/O command non-live; output tạo mới để không ghi đè.
<code>run_live_capacity_measure()</code>	Annex-B stdin → JSON metrics stdout	Parse stream và đếm IDR/candidate/capacity.
<code>run_live_authenticated_embed()</code>	stdin chứa key, payload rồi Annex-B → Annex-B stego stdout	Incremental reader + stream encoder; xử lý từng NAL/IDR.
<code>run_live_authenticated_extract()</code>	Key line + Annex-B stdin → payload hex stdout	Incremental reader + stream decoder; kết thúc khi payload authenticate.
<code>set_standard_streams_to_binary()</code>	stdin/stdout handles → binary mode	Bắt buộc trên Windows để CRLF/encoding text không đổi bytes H.264.
<code>parse_bit_count()</code>	decimal string → size_t	Reject số âm, overflow, ký tự thừa.

`AnnexBNalStreamReader::read_next()` giữ buffer hữu hạn quanh một NAL và một input chunk, nhận cả start code đi qua ranh giới chunk; giới hạn NAL tối đa mặc định 16 MiB. Đây là cơ chế giảm yêu cầu giữ toàn bộ raw input trong live WebSocket, nhưng không đồng nghĩa toàn bộ API/circuit/video analytics đều có memory bounded như nhau.

7 Verifier: từ video stego đến quyết định accept/reject

7.1 Blind extraction native

`demo/07_blind_extract.py::main()` đọc `stego_video.h264`, lấy key từ session, gọi `zkstego_blind_bitsextract-stream-auth`. Cover video không được truyền cho lệnh này. Output stdout hex được chuyển về bytes và ghi `extracted_payload.bin`. Native decoder:

1. cập nhật SPS/PPS gần nhất;
2. nhận IDR NAL, dựng segment cho stream decoder;
3. decode lại CAVLC candidates, xếp theo keyed schedule, gom sign bits;
4. đọc frame length/version, đợi đủ dữ liệu, xác thực HMAC;
5. trả payload bytes hoặc lỗi nếu stream hết/sai key/sai tag.

7.2 Unpack và verify Groth16

`unpack(blob)` kiểm tra tối thiểu 4 byte đầu, đọc độ dài message big-endian, kiểm tra blob đủ $4 + |m| + 129$ byte rồi tách $(m, proofBytes)$. Trong demo, `08_unpack_payload.py` ghi hai output riêng `recovered_message.bin` và `recovered_proof_129_bytes.bin`.

Tiếp theo `bytes_to_proof()` tái dựng điểm Groth16. `ZKSnarkBridge._build_public_signals(message, key)` tính hash message, commitment và length theo circuit. `verify()` gọi `snarkjs` với `verification_key.json`; `_snarkjs_verify()` tạo temporary public/proof JSON rồi đọc exit/result. `verify_proof_for_payload()` là wrapper thuận tiện dựng public signals từ payload/key trước khi gọi `verify`.

Trong demo 09, cùng proof với message gốc phải valid; nới thêm một byte ! phải invalid. Đây là test negative hữu ích, nhưng không kiểm tra mọi kiểu tamper vào H.264, mọi public statement hoặc quản lý key trong production.

7.3 HTTP và WebSocket làm gì, không làm gì

`create_native_app()` trong `src/api/native_handlers.py` dựng app bằng `create_app()` nhưng đưa `verify_handler=None`. HTTP embed nhận cover, message bytes và secret key; HTTP extract nhận stego cùng key. Endpoint `verify native` vì thế trả 501. Nếu muốn embed Groth16, caller phải tự tạo proof-bearing blob trước và truyền blob đó như `message`; native layer coi payload là opaque bytes.

WebSocket `/api/v1/stream` nhận control JSON đầu tiên. Embed control chứa operation, key và message base64; sau đó client đẩy các binary chunk Annex-B và gửi control `"type": "end"`. Server mở native process, forward chunk qua stdin và output qua WebSocket với flow-control/backpressure. Extract control chứa operation, key và maximum payload bytes; output là extracted bytes. `native_stream()` giới hạn thời gian, tổng byte, kích thước chunk, số chunk, stdin high/low-water mark và lấy mẫu RSS/CPU/latency. Đây là service streaming trên elementary H.264; input camera device/capture driver chưa tự được mở bởi module này.

8 API chung: hàm nào nhận request, hàm nào chạy job

`src/api/app.py::create_app()` đăng ký endpoints và ghép các handler được inject. Các khối hàm chính:

Hàm/lớp	Input → output	Ảnh hưởng
<code>RequestBodyGuard.__call__()</code>	ASGI receive/send guarded request →	Reject body quá giới hạn hoặc timeout trước khi parse upload.
<code>WorkDirLease.acquire/release()</code>	work directory → lease lifecycle	Ngăn nhiều service instance dùng chung work directory trái phép.
<code>ApiSettings.from_environment()</code>	environment → settings	Nạp token, paths, queue/upload/time limits.
<code>JobStore.create/get/update/discard()</code>	job record ↔ JSON on disk	Job ID/status persist để client poll và download.
<code>_decode_key()</code> / <code>_decode_message()</code>	base64 form field → bytes	Kiểm format/độ dài trước gọi native handler.
<code>_save_upload()</code>	UploadFile → file bounded-size	Ghi video input theo giới hạn request; lỗi sẽ cleanup.
<code>require_token()</code>	Authorization header → pass/401	API bearer authentication, không phải key CAVLC/Groth16.
<code>reserve_capacity()</code>	semaphore → slot hoặc reject	Chặn vượt số worker/job queue cấu hình.
<code>submit()</code>	job id + operation + callable → worker thread, cập nhật state	Chạy tác vụ nền; job endpoint trả 202 rồi client poll.
<code>startup()/shutdown()</code>	service lifecycle → recovery/cleanup	Khôi phục job đang dở, khởi động/dừng cleanup worker.
<code>start_embed()</code>	upload cover + message/key → accepted job	Lưu input, tạo job và gọi injected embed handler.
<code>start_extract()</code>	upload stego + key/bounds → accepted job	Gọi extractor và lưu payload artifact để download.
<code>start_verify()</code>	stego + cover + key/length → job hoặc 501	Chỉ chạy nếu verify handler được inject; native adapter cố ý không inject.
<code>get_job()</code> / <code>download_artifact()</code>	job id → status/file response	Poll và tải kết quả; cleanup/policy quyết định thời gian tồn tại.

Trong `native_handlers.py`, `invoke()` đưa key vào stdin child process, không đặt key trên command line hay key temp file. `embed_handler()` gọi `embed-stream-auth-stdin`; `extract_handler()` gọi `extract-stream-auth` rồi validate hex, giới hạn payload và tạo file mới. `native_stream()` là đường WebSocket độc lập với queue HTTP; nó quản lý child process, tasks cho stdin/stdout, cancellation, deadlines và metrics.

9 Nhánh Python full-video cũ: các hàm và phụ thuộc

9.1 Không nhầm nhánh này với native blind

`src/embedder.py::embed()` là pipeline Python đã tồn tại song song. Nó cần cover để phân tích vị trí, tạo proof bên trong hàm, gói payload, lấy safe positions, patch coefficients qua Python reconstruction và ghi sidecars. Hàm `src/verifier.py::verify()` yêu cầu cả stego lẫn original cover, trừ khi caller cung cấp vị trí đã tính trước. Demo 13 gọi nhánh này riêng trên operating contract benchmark khác.

9.2 Các helper của embedder

Hàm trong <code>src/embedder.py</code>	Input → output	Consumer
<code>_get_bridge(circuits_dir)</code>	path → cached bridge	Dùng lại ZK wrapper cho lần embed tiếp theo.
<code>_prune_patchable_positions()</code>	safe positions + offsets → chỉ candidate block có thể patch	Ngăn yêu cầu vượt quá patchability thật của bitstream.
<code>_limit_positions_per_block()</code>	positions, cap → bounded schedule	Thực thi giới hạn sửa mỗi block để giảm artifact/duplicate.
<code>_filter_reconstructed_positions()</code>	planned/applied block sets → thật sự được output áp dụng	Không báo “đã nhúng” cho patch mà reconstructor bỏ qua.
<code>_resolve_video_analysis()</code>	video hoặc precomputed tuple → 6 phần phân tích	Ưu tiên tuple caller, nếu không nạp/build cache.
<code>embed()</code>	video, message, key, circuit path, options → EmbedResult + H.264 + sidecars	Orchestrator của Python legacy branch.

Bên trong `embed()`, trình tự chính là: `validate file/message/key`; `analyze_stream_profile()`; sinh Groth16 proof; nén proof; `pack()`; (tùy chọn) chaos scramble; `resolve analysis/cache`; lọc safety/patchability; gọi `PayloadEmbedder.embed_payload()`; gọi `BitstreamReconstructor.reconstruct_video()`; lọc lại vị trí theo `applied_block_keys`; thử tối đa 12 vòng loại các block không tái dựng được; lưu video, `.positions.json`, `.meta.json`, `.manifest.json`; trả `EmbedResult`. Dữ liệu quan trọng trong result gồm bits yêu cầu/đã nhúng, capacity từng lớp, output path, proof/public dict và used positions.

9.3 Phân tích H.264/CAVLC của Python path

Hàm/lớp	Input → output	Ý nghĩa trong chuỗi
<code>NALParser.parse()</code>	H.264 bytes → NALUnit list	Tách start codes, header và Ebsp để các parser cấp sau làm việc.
<code>_find_start_codes()</code>	bytes → [(offset,size)]	Tìm phân cách Annex-B 3/4 byte.

Hàm/lớp	Input → output	Ý nghĩa trong chuỗi
<code>_extract_nal_unit()</code>	offset range → NALUnit	Lấy type/ref idc và loại emulation-prevention bytes cho RBSP.
<code>SliceHeaderParser.parse()</code>	reader + NAL/SPS/PPS → SliceHeader	Đọc các Exp-Golomb/slice flags cần để đến macroblock data.
<code>MacroblockParser.parse_macroblock()</code>	reader + slice context → MacroblockData	Đọc mb type, prediction/CBP và phân ranh residual syntax.
<code>MacroblockParser.calculate_nC()</code>	tọa độ block + neighbour counts → n_C	Context này chọn đúng coeff token table của CAVLC.
<code>TraceableCAVLCParser.extract_with_offsets()</code>	IDR NAL + SPS/PPS → block levels, offsets, MB indices	Cấp cho safety filter/patcher biết bit nào mã hóa hệ số nào.
<code>CAVLCDecoder.decode_block_cavlc()</code>	bit reader + n_C + max coeff → CoefficientBlock	Giải coeff_token, trailing signs, levels và zero runs thành coefficient list.
<code>_decode_coeff_token()</code>	n_C + reader → TotalCoeff, TrailingOnes	TrailingOnes quyết định sign bits được đọc trước levels.
<code>_decode_levels()</code>	count + trailing ones → level list	Đọc level code sau sign prefix.
<code>_decode_total_zeros()</code> <code>_decode_runs()</code>	/ block residual syntax → positions of nonzero coeff	Giữ đúng scan order khi reconstruct block.
<code>CAVLCEncoder.encode_block_cavlc()</code>	coefficient list + n_C → VLC bits	Legacy reconstruction reserializes modified blocks.
<code>BitstreamReader.read_ue/read_se()</code>	RBSP bits → unsigned/signed Exp-Golomb value	Dùng cho syntax H.264 như ue(v), se(v).

9.4 Safety filter, patch và tái dựng Python

Trong `src/core/stego.py`, `CAVLCSafetyFilter._detect_trailing_ones()` tìm index trailing-one; `_verify_block_bit_length_invariance()` kiểm rằng ứng viên không làm đổi bit length theo tiêu chí patch; `get_safe_positions()` lấy tập an toàn rộng hơn; `get_safe_sign_positions()` giới hạn vào sign positions phù hợp. `sort_blocks_interleaved()` phân bố thứ tự theo block/MB để tránh dồn sửa một vùng.

`PayloadEmbedder.embed_payload()` chuyển blob thành bit, cấp từng bit cho candidate và gọi `_embed_with_safety_filter()`; output là danh sách coefficient block đã đổi, số bit thực nhúng và vị trí đã dùng. Chiều đọc là `extract_payload()` / `_extract_with_safety_filter()`, lấy sign từ coefficient theo đúng position order rồi `_bits_to_bytes()`. Hàm `_modify_lsb()` thực chất sửa dấu/LSB theo quy tắc block hiện hành; không nên hiểu là sửa pixel LSB trong ảnh RGB.

`BitstreamPatcher.patch_slice()` áp các modification vào NAL theo offset; `validate_block_patchability()` xác nhận vị trí/độ dài block; `_encode_coefficients_to_bits()` tái mã hóa CAVLC block. Sau đó `BitstreamReconstructor.reconstruct_video()` cập nhật NALs và gọi `_reconstruct_slice_with_cavlc()` / `_write_h264_file()` để tạo file mới; SPS/PPS parser duy trì context; emulation prevention được thêm lại. Các validator FFmpeg/PSNR là tùy chọn, không đồng nghĩa mọi lần chạy embed mặc định đều đã chứng minh quality.

9.5 Verifier Python yêu cầu những gì

`src/verifier.py::verify()` validate cả `stego_video_path` và `original_video_path`, circuit path, 32-byte key và message length. Nó đọc sidecar qua `_load_sidecar_data()` nếu có; nếu không thì tái phân tích cover để tạo position map. Tiếp đó tính số bit cần, dùng CAVLC analysis, áp chaos inverse nếu bật, đọc bit bằng `extract_bits_direct()`, gọi `unpack()`, `bytes_to_proof()`, dùng public signals rồi `bridge.verify()`. Output `VerifyResult` gồm valid, message (chỉ giữ khi hợp lệ), proof/public dict và bits extracted.

`verifier_blind_keyed.py::verify_blind_keyed()` là một thử nghiệm riêng không sidecar: lấy message length để ước lượng fixed payload bits, gọi `derive_blind_positions_validated_pool_proxy()`, parse stego, trích block, đọc bits, unpack và verify. Nó nêu assumption cố định/no-chaos; không nên đánh đồng nhất với native stream decoder vốn có format/frame/selector contract khác.

10 Cache, scheduler và những nhánh hỗ trợ

10.1 Cache phân tích video

`src/core/analysis_cache.py::_video_fingerprint()` tạo dấu vân tay đầu vào; `_code_fingerprint()` gắn cache với fingerprint source; `_cache_path()` chọn file; `load_or_build_video_analysis()` chạy NAL/CAVLC analysis và safety filter hoặc nạp trusted pickle; `_build_reconstruction_context()` chuẩn bị dữ liệu reconstructor; `load_or_build_reconstruction_context()` đọc/build context riêng; `clear_video_analysis_cache()` xóa các pickle cache.

Cache giảm thời gian lặp lại nhưng pickle chỉ an toàn nếu nguồn cache đáng tin cậy. Biến `ZK_STEGO_TRUSTED_PICKLE_CACHE=1` là opt-in rõ ràng ở nhánh Python demo; không bật với cache nhận từ nguồn lạ. Native CLI demo không dùng Python pickle analysis cache.

10.2 Realtime scheduler abstraction

`CAVLCRealtimeBudget.__post_init__()` validate FPS/queue/memory bounds; `frame_interval_s` trả $1/fps$; `latency_limit_s` chọn p95 limit hoặc frame interval. `RealtimeCAVLCScheduler.submit()` validate segment size, epoch/cache và queue bound; `process_next()` lấy segment + proof callback, gọi patch callback, đo latency; `report()` tính processed/dropped/p95 và accepted. Đây là orchestration/test abstraction; implementation comment chủ động cấm dùng full-video Python embed làm patcher realtime. Native WebSocket hiện gọi native process trực tiếp và có cơ chế riêng.

10.3 Key certificate và manifest

`src/key_policy.py::issue_key_certificate()` ký Ed25519 certificate có subject public key, key id, issuer, not-before và expires-at; `KeyCertificate.verify()` kiểm chữ ký issuer, format/version và cửa sổ thời gian; `verify_manifest_with_certificate()` kiểm certificate trước, rồi dùng subject public key để verify manifest. `src/manifest.py::StegoManifest.sign(), verify_signature(), compute_content_hash(), to_json()/from_json()` quản lý metadata sidecar.

Các helper này chỉ enforce expiry nếu đúng verifier được gọi ở mọi điểm chấp nhận key. Demo native hiện cấp trực tiếp một raw 32-byte key và không gọi `KeyCertificate.verify()`; vì vậy certificate module không tự làm cho raw key native hết hạn.

11 Demo đã chạy: input cụ thể và cách suy luận

Session kiểm chứng gần nhất nằm ở `demo/runs/session_20260925T033640Z/`. Cover là `data/encoded/foreman_cif_q18_g1_300f.h264`; message 15 byte; proof nén 129 byte; tất cả stage log 00–13 báo PASS trong `SUMMARY.txt`.

Artifact/log	Giá trị thực tế	Có thể suy ra / chưa thể suy ra
01_generate_proof.log	129 B proof, 513 public signal scalars	Proving chạy với setup hiện có; không chứng minh key/proving key được quản lý production.
02_pack_payload.log	15 B message + 129 B proof + 4 B length = 148 B	Outer payload format đúng, 1.184 bit trước native frame overhead.
04_candidate_capacity.log	300 IDR; 105,516 raw sign candidates; capacity bị cap 19,200 bit	Fixture có nhiều candidate hơn yêu cầu 1,336 bit; không đại diện mọi H.264/GOP.
04_first_candidate_location.log	NAL 3, MB 0, RBSP bit 91	Parser tìm được một sign sample có địa chỉ; file probe chỉ minh họa một bit.
05_embed_video.log	148 B payload; H.264 output 3,551,610 B	Native embed hoàn tất với fixture; kích thước output giống input trong sample.
06_ffmpeg_decode.log	FFmpeg -xerror exit 0	Stream giải mã được; không suy ra PSNR/SSIM hay invisibility.
07_blind_extract.log	Trích được 148 B, không cấp cover file	Blind extraction hoạt động cho fixture/key này.
09_verify_proof.log	Original message valid; message sửa một byte invalid	Proof verification và negative case chạy; không kiểm mọi loại statement/tamper.
10_wrong_key_rejection.log	Exit 2, authenticated frame version invalid	Sai key bị reject; lỗi xảy ra ở native frame/extraction chứ không phải Groth16.
11_http_transport.log	HTTP embed/extract round-trip; verify endpoint 501 đúng thiết kế test	Channel vận chuyển payload hoạt động; native server chưa verify Groth16.
12_websocket_e2e.log	5/5 native channel E2E tests PASS	Loopback tests xác nhận protocol/process path, không phải thiết bị camera edge production.
13_python_pipeline.log	Nhánh Python verify PASS, 1,176 bit; output 16,768,616 B trên deadline 600-frame input	Chứng minh nhánh legacy hoạt động với benchmark contract khác; không phải so sánh cùng nguồn hay performance công bằng.

11.1 Artifacts nên giữ khi debug

- `session.json`, `00_prepare.log`: xác nhận input đúng và fingerprint có thể tái lập.
- `candidate_capacity.json`, `04_positions_summary.txt`: kiểm tra payload có vượt sức chứa hay không.
- `stego_video.h264`: output chính của đường native; so hash với `http_stego_video.h264` khi

kiểm tra HTTP artifact.

- `extracted_payload.bin`, `recovered_message.bin`, `recovered_proof_129_bytes.bin`: xác định lỗi nằm ở extraction, framing, unpack hay ZK verify.
- `SUMMARY.txt`: trạng thái tổng hợp. Luôn mở log gốc để xem vì sao step fail; summary chỉ phân loại theo marker PASS/exit code.

`secret_key.bin` là dữ liệu nhảy cảm, dù là key demo. Không commit, gửi người khác hoặc đưa vào log. `wrong_key.bin` chỉ phục vụ negative test.

12 Các giới hạn cần nhớ khi đọc code

1. **Không phải mọi H.264 đều được hỗ trợ như nhau.** Native path nhắm Baseline/CAVLC và các điều kiện parser/slice cụ thể; CABAC, FMO, MBAFF và nhiều trường hợp profile khác nằm ngoài primitive đã mô tả.
2. **I/P/B và nhúng là hai lớp khác nhau.** Parser native thu candidate từ IDR I-slice CAVLC; demo fixture all-intra khiến mỗi frame phù hợp là IDR. Đừng suy ra từ đó rằng mọi P/B residual hiện đều có thể làm carrier.
3. **Blind không có nghĩa không cần key.** Blind có nghĩa extractor không cần cover gốc; vẫn cần key để đồng bộ lịch và authenticate frame.
4. **Groth16 và HMAC khác nhiệm vụ.** Proof kiểm relation circuit; HMAC kiểm frame payload theo shared key; bearer token xác thực API caller; các lớp không thay thế nhau.
5. **Native streaming và camera realtime không đồng nghĩa.** WebSocket nhận byte stream Annex-B và có bounded I/O; camera capture/device driver và chứng minh deadline trên edge cần test riêng.
6. **Nhánh Python sinh sidecar.** `.positions.json`, `.meta.json`, `.manifest.json` là artifact của nhánh cũ, không cần cho native blind extraction.

13 Lộ trình đọc source cho người mới tiếp nhận

Để nắm hệ thống mà ít bị phân tán, nên đọc theo thứ tự sau:

1. Chạy/đọc `demo/README.md`, rồi theo các script 00–09. Mở mỗi log ngay sau khi chạy để liên hệ input/output.
2. Đọc `demo/05_embed_video.py` và `demo/07_blind_extract.py` để thấy Python chỉ làm orchestration ở luồng chính.
3. Đọc `src/api/native_handlers.py::embed_handler()`, `extract_handler()` và `native_stream()` để nắm boundary HTTP/WS.
4. Đọc CLI `native/tools/blind_bits.cpp::main()`, sau đó `run_live_authenticated_embed()` / `run_live_authenticated_extract()`.
5. Đọc C++ library theo chuỗi `split_annex_b()` → `SPS/PPS/slice parsing` → `decode_baseline_i_idr_slices()` → `collect_cavlc_trailing_one_sign_candidates()` → `select_keyed_cavlc_sign_candidates()` → `AuthenticatedCavlcStreamEncoder/Decoder`.
6. Đọc proof path: `src/zk_proof.py::pack()`, `proof_to_bytes()`, `ZKSnarkBridge` và `circuits/payload_verify.circom`.
7. Cuối cùng đọc `src/embedder.py/src/verifier.py` để hiểu vì sao nhánh Python khác native; đọc `demo/13_python_pipeline.py` như một integration test/benchmark path riêng.

14 Tra cứu nhanh theo câu hỏi debug

Triệu chứng	Nơi bắt đầu lần theo
Không thấy video NAL/IDR	demo/03_read_video.log, C++ split_annex_b(), decode_baseline_i_idr_slices(), SPS/PPS parser.
Capacity thấp hoặc payload không fit	04_candidate_capacity.log, collect_cavlc_trailing_one_sign_candidates(), max bits mỗi IDR.
Embed trả nonzero hoặc output thiếu	05_embed_video.log, run_live_authenticated_embed(), encoder process_segment()/complete().
FFmpeg từ chối video	06_ffmpeg_decode.log; xác định parser/patch/EBSP boundary trước khi kiểm quality.
Extract báo sai version/tag	So key bytes hai phía, cùng native schedule/frame domain, sau đó AuthenticatedCavlcStreamDecoder::consume_segment().
Payload extract được nhưng ZK false	So packed_payload.bin với output unpack; kiểm 4-byte length, proof 129 B, message/key/public signals và verification key.
HTTP status 501 cho verify	Đúng với native adapter hiện tại: create_native_app(...verify_handler=None). ZK verify phải chạy riêng.
Python chạy lâu/RAM cao	Nhánh src/embedder.py, parser toàn-video, analysis cache và demo/13_python_pipeline.py; không quy kết cho native CLI.
Khóa hết hạn vẫn dùng được	Kiểm tra có gọi KeyCertificate.verify() tại enforcement boundary hay chỉ cấp raw key cho native. Hiện demo path cấp raw key.

15 Kết luận vận hành

Khi đọc một đường lỗi, lần theo artifact tuần tự: đầu vào/session fingerprint, proof bytes, packed blob, capacity, stego output, extracted blob, unpacked message/proof, rồi Groth16 result. Điểm nổi quan trọng nhất là native CLI nhận payload bytes opaque; Python demo tạo proof và blob, còn H.264 syntax parsing, keyed sign schedule, HMAC frame, patch, blind extraction thuộc C++.

Đường native đã được chạy end-to-end trên fixture hỗ trợ, nhưng source vẫn có nhánh Python legacy, scheduler abstraction, certificate/manifest helpers và trust experiments không tự động nằm trong đường runtime đó. Khi thay đổi code, hãy kiểm tra đúng consumer kế tiếp của hàm vừa sửa và chạy test ở boundary tương ứng; PASS ở một nhánh không xác nhận các nhánh còn lại.